

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Відокремлений структурний підрозділ

«Волинський фаховий коледж

Національного університету харчових технологій»

## **КУРСОВА РОБОТА**

з дисципліни «Об'єктно-орієнтоване програмування»

**на тему: «Облікова система музичної школи»**

**Виконав:** студент 3 курсу, групи ПЗ-32

Слушаєв М. О.

**Перевірила:**

Шепелюк Г. С.

## ЗМІСТ

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Вступ .....                                                       | 3  |
| Розділ 1. Теоретична частина .....                                | 4  |
| 1.1 Технічне завдання на розробку системи .....                   | 4  |
| 1.2 Опис технологій розробки інформаційної системи .....          | 5  |
| 1.3 Опис технології зберігання даних .....                        | 7  |
| Розділ 2. Практична частина .....                                 | 9  |
| 2.1 Інфологічна модель предметної області .....                   | 9  |
| 2.2 Архітектура застосунку .....                                  | 10 |
| 2.3 Структура підсистеми зберігання даних .....                   | 12 |
| 2.4 Структура модулів програми .....                              | 13 |
| 2.5 Опис інтерфейсу користувача .....                             | 19 |
| 2.6 Результати тестування роботи програми .....                   | 20 |
| Висновки .....                                                    | 23 |
| Список використаних джерел .....                                  | 24 |
| Додатки .....                                                     | 25 |
| Додаток А. Код модуля <code>models.py</code> (фрагмент) .....     | 25 |
| Додаток Б. Код модуля <code>repository.py</code> (фрагмент) ..... | 26 |
| Додаток В. Структура проєкту .....                                | 27 |

## ВСТУП

Актуальність теми. Музичні школи є важливими осередками художньої освіти, проте облік учнів, викладачів, занять та успішності часто ведеться паперово або лише частково автоматизовано. Це призводить до зайвих витрат часу, помилок у документації та ускладнює підготовку звітності.

Мета курсової роботи — розробити консольну облікову систему музичної школи, яка автоматизує роботу з даними про учнів, викладачів, предмети, групи, заняття та оцінки і водночас демонструє принципи об'єктно-орієнтованого програмування.

Завдання роботи:

- проаналізувати предметну область та сформулювати вимоги до системи;
- спроектувати доменну модель з урахуванням принципів ООП (наслідування, поліморфізм, інкапсуляція, абстракція);
- розробити архітектуру програми на основі патерну Repository;
- реалізувати CLI-інтерфейс за допомогою бібліотеки Click;
- забезпечити збереження даних у реляційній базі SQLite через SQLAlchemy 2.0;
- написати автоматизовані тести для ключових функцій системи.

Практична цінність. Система дає змогу швидко вносити, редагувати та переглядати дані, формувати звіти про успішність і наповнювати базу демонстраційними даними для перевірки роботи програми.

## РОЗДІЛ 1. ТЕОРЕТИЧНА ЧАСТИНА

### 1.1 Технічне завдання на розробку системи

**Мета розробки.** Створити інформаційну систему для автоматизації обліку навчального процесу музичної школи. Система повинна спростити ведення відомостей про учнів, викладачів, навчальні групи, заняття та результати навчання.

#### **Функціональні вимоги:**

- управління даними учнів (додавання, перегляд, редагування, видалення);
- управління даними викладачів зі спеціалізацією;
- ведення довідника музичних предметів;
- формування груп учнів та прив'язка їх до занять;
- реєстрація занять із зазначенням дати, часу, аудиторії, викладача та групи;
- виставлення оцінок учням за конкретне заняття за 12-бальною шкалою;
- формування звітної інформації про успішність учнів та навчальних предметів;
- забезпечення зручного перегляду та пошуку даних.

#### **Нефункціональні вимоги:**

- кросплатформенність — Windows, Linux, macOS;
- мінімальні системні вимоги: оперативна пам'ять 512 МБ, вільне місце на диску 100 МБ;
- час виконання типових операцій повинен бути достатнім для комфортної роботи користувача;
- дані повинні зберігатися без втрати цілісності та бути доступними після повторного запуску програми.

**Вимоги до інтерфейсу.** Інтерфейс повинен бути зрозумілим для користувача, забезпечувати логічний поділ команд за призначенням, а повідомлення про результати операцій і помилки мають бути однозначними та читабельними.

**Архітектура програми.** Система повинна мати модульну будову з розділенням рівня інтерфейсу, бізнес-логіки, моделі даних та підсистеми зберігання.

**Зберігання даних.** Інформація про навчальний процес повинна зберігатися у структурованому вигляді з підтримкою зв'язків між основними сутностями.

**Тестування та валідація.** Потрібно забезпечити перевірку коректності основних сценаріїв роботи системи, обробки помилкових введень і достовірності розрахованих результатів.

## **1.2 Опис технологій розробки інформаційної системи**

Для реалізації курсової роботи обрані наступні технології та інструменти.

**Python 3.12** — основна мова програмування. Підтримує типові анотації, `dataclasses`, модуль `abc` для абстрактних класів і сучасний синтаксис `match/case`. Вільно поширюється та має велику екосистему бібліотек.

**SQLAlchemy 2.0** — ORM для Python. У версії 2.0 використовується сучасний стиль оголошення моделей через `DeclarativeBase` та `Mapped[ ]`, що дозволяє описувати таблиці бази даних у вигляді Python-класів і працювати з ними через об'єктну модель.

**SQLite** — вбудована реляційна СУБД, яка зберігає дані в єдиному файлі. Не потребує окремого серверного процесу, що спрощує розгортання навчальної системи. Підтримує стандарт SQL та зовнішні ключі.

**Click** — бібліотека для створення CLI-інтерфейсів у Python. Дозволяє описувати команди, параметри та їх типи за допомогою декораторів, автоматично генерує довідку (`--help`) і перевіряє введені дані.

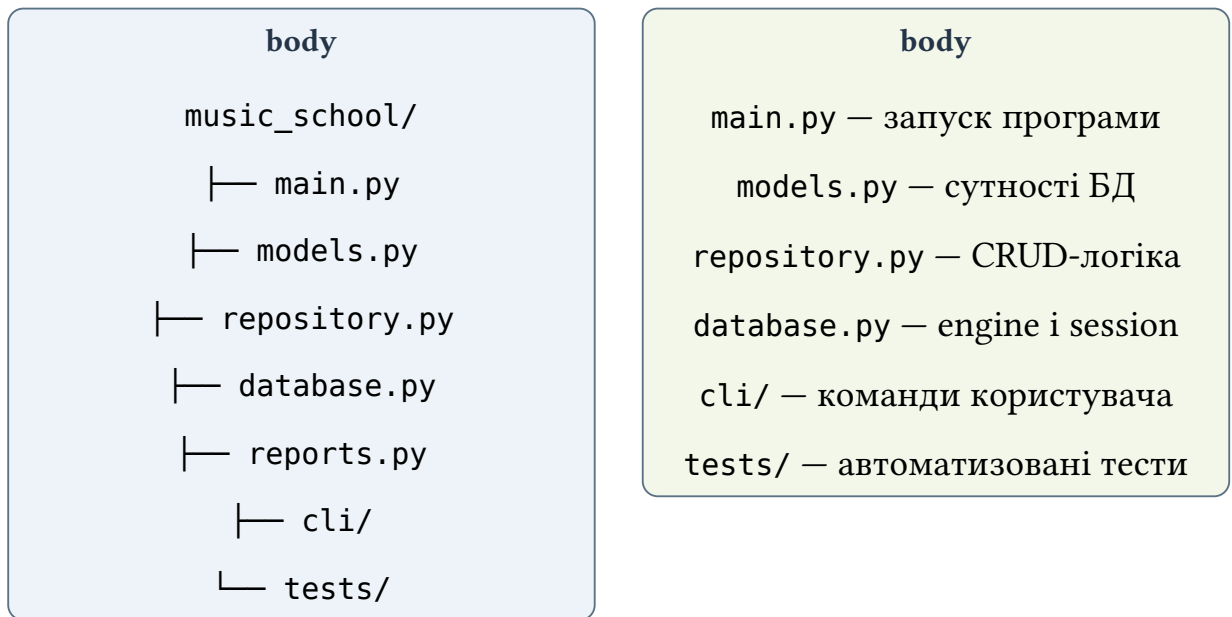
**uv** — сучасний менеджер пакетів та середовищ Python. Значно швидший за `pip`, дозволяє відтворювати оточення через файл `uv.lock`.

**unittest** — стандартна бібліотека Python для написання та запуску автоматизованих тестів.

**Typst** — система верстки документів, використана для автоматизації оформлення курсової роботи. Вона дає змогу уніфікувати структуру документа, стилі заголовків, таблиці, підписи до рисунків і повторювані елементи оформлення.

**Claude** та **Codex** — допоміжні інструменти, які використовувалися під час підготовки шаблону документа, створення однотипних фрагментів Typst-коду, стилізації схем і редагування тексту. Остаточне наповнення, перевірка змісту та приведення роботи до вимог виконувалися автором.

Обраний стек покриває всі основні задачі системи: зберігання даних, командний інтерфейс і тестування.



Таблиця 1: Графічна схема структури проєкту `music_school`

### 1.3 Опис технології зберігання даних

**Реляційна база даних.** Для зберігання інформації обрано реляційну модель даних на основі SQLite. Вона передбачає організацію даних у вигляді таблиць, пов'язаних між собою зовнішніми ключами.

**SQLite** — реляційна СУБД у вигляді бібліотеки. Дані зберігаються в одному файлі (`school.db`). Її перевагами є відсутність потреби в окремому сервері, простота резервного копіювання та підтримка транзакцій.

**SQLAlchemy 2.0 ORM.** Взаємодія з базою відбувається через Python-об'єкти, а не через сирі SQL-запити. Це зменшує кількість помилок, спрощує підтримку коду і дає змогу за потреби змінити СУБД без повної переробки моделі.

**Сесії та транзакції.** У системі використовується `Session` від SQLAlchemy, яка об'єднує операції в транзакції. Після успішного виконання змін виконується `commit()`, а у випадку помилки — `rollback()`. Це забезпечує цілісність даних.

Таблиця 1.1 — Таблиці бази даних та їх призначення

| Таблиця        | Призначення                                        | Кількість полів |
|----------------|----------------------------------------------------|-----------------|
| students       | Облік учнів: ім'я, телефон, дата народження        | 4               |
| teachers       | Облік викладачів: ім'я, телефон, спеціалізація     | 4               |
| subjects       | Довідник музичних предметів                        | 2               |
| groups         | Групи учнів                                        | 2               |
| group_students | Зв'язок many-to-many: група ↔ учень                | 2               |
| lessons        | Заняття: дата, аудиторія, викладач, група, предмет | 6               |
| grades         | Оцінки учнів за заняття (1–12 балів)               | 5               |

## РОЗДІЛ 2. ПРАКТИЧНА ЧАСТИНА

### 2.1 Інфологічна модель предметної області

**Предметна область.** У межах музичної школи ведеться облік учнів, викладачів, предметів, груп, занять і результатів навчання.

#### Основні сутності предметної галузі:

- **Учень (Student)** — особа, що навчається у школі. Характеризується ім'ям, датою народження та контактним телефоном. Учень може входити до кількох груп та отримувати оцінки.
- **Викладач (Teacher)** — особа, що проводить заняття. Має ім'я, телефон та музичну спеціалізацію (фортепіано, гітара, вокал тощо). Викладач може вести кілька занять.
- **Предмет (Subject)** — навчальна дисципліна (наприклад, «Фортепіано», «Сольфеджіо»). Має унікальну назву.
- **Група (Group)** — об'єднання учнів для спільного навчання. Група має назву і містить кількох учнів (зв'язок many-to-many).
- **Заняття (Lesson)** — конкретний урок у розкладі. Прив'язане до викладача, групи, предмета, має дату/час та аудиторію.
- **Оцінка (Grade)** — результат навчання учня на конкретному занятті. Значення від 1 до 12 балів із необов'язковим коментарем.

#### Зв'язки між сутностями:

Таблиця 2.1 — Зв'язки між сутностями

| Від              | Тип          | До                 |
|------------------|--------------|--------------------|
| Group (група)    | many-to-many | Student (учень)    |
| Lesson (заняття) | many-to-one  | Teacher (викладач) |
| Lesson (заняття) | many-to-one  | Group (група)      |
| Lesson (заняття) | many-to-one  | Subject (предмет)  |

|                |             |                  |
|----------------|-------------|------------------|
| Grade (оцінка) | many-to-one | Student (учень)  |
| Grade (оцінка) | many-to-one | Lesson (заняття) |

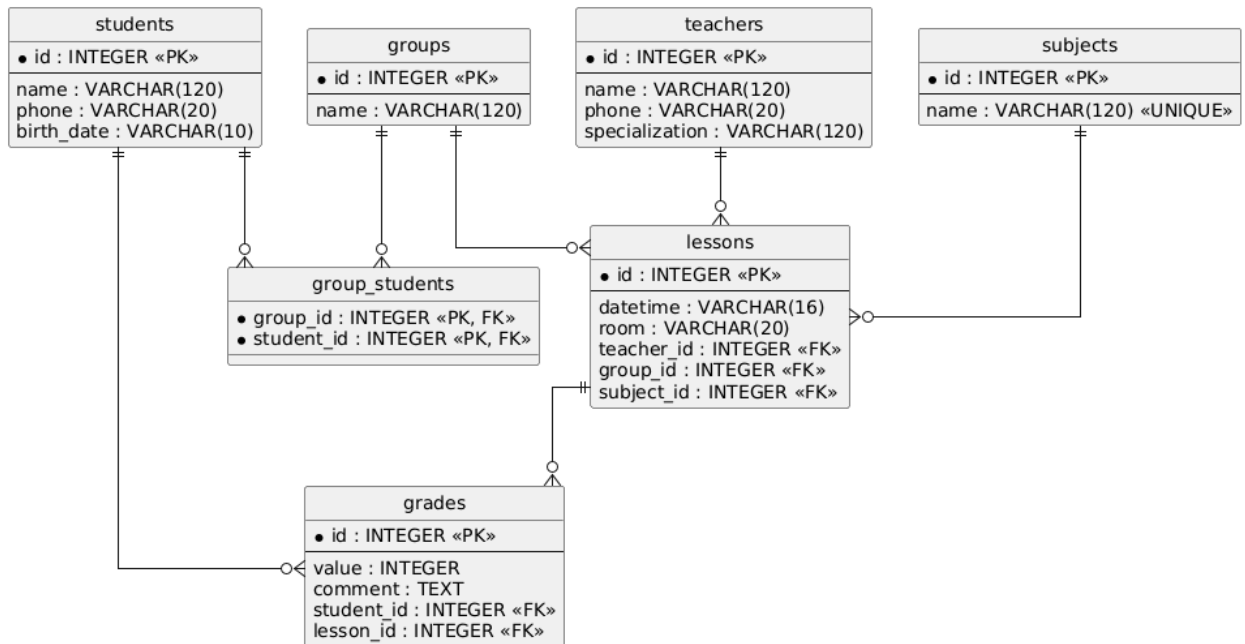


Рисунок 1: ER-схема предметної області облікової системи музичної школи

## 2.2 Архітектура застосунку

Застосунок побудовано за багатошаровою архітектурою.

**Рівень 1 — Презентація (CLI).** Каталог `cli/` містить групи команд, а `main.py` реєструє їх у середовищі Click.

**Рівень 2 — Бізнес-логіка.** `repository.py` відповідає за доступ до даних, а `reports.py` — за обчислення і звіти.

**Рівень 3 — Доменна модель.** `models.py` описує ORM-класи та їх зв'язки.

**Рівень 4 — Інфраструктура.** `database.py` налаштовує SQLAlchemy, керує сесіями та забезпечує початкове наповнення даними.

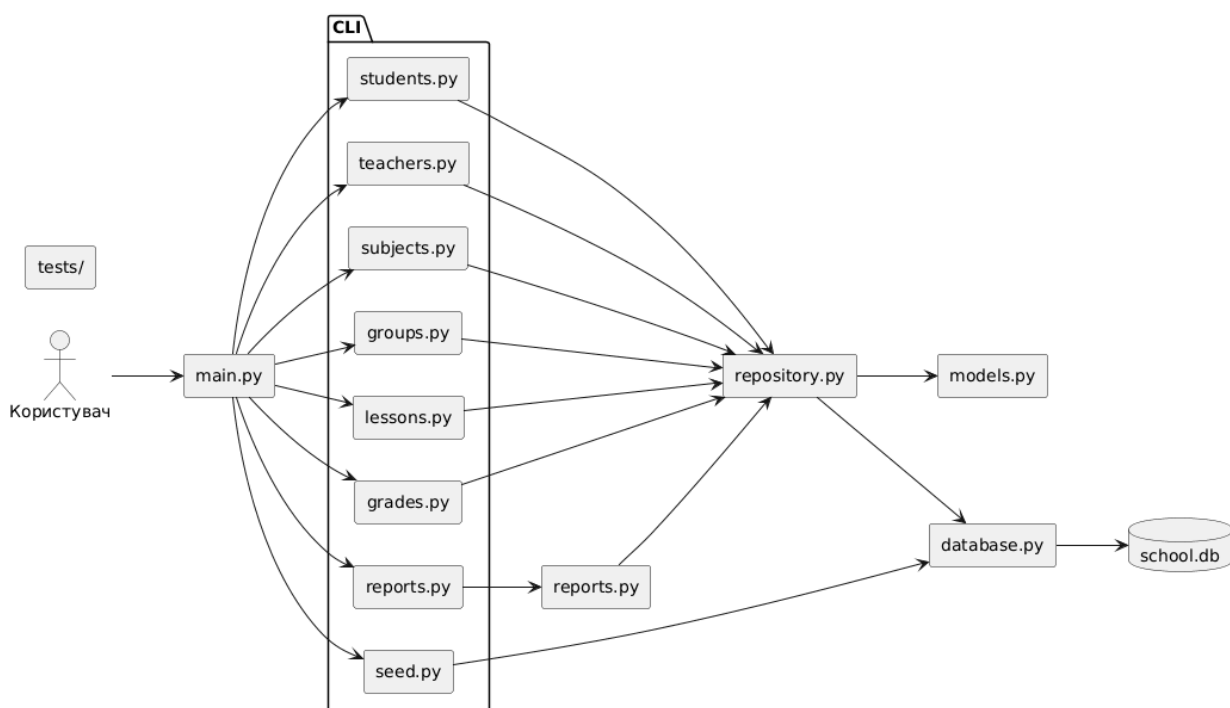


Рисунок 2: Діаграма компонентів облікової системи музичної школи

Таблиця 2.2 — Модулі проєкту та їх призначення

| Файл / Каталог | Призначення                                                                     |
|----------------|---------------------------------------------------------------------------------|
| main.py        | Точка входу CLI; реєстрація груп команд                                         |
| models.py      | ORM-моделі доменних сутностей (Student, Teacher, Subject, Group, Lesson, Grade) |
| repository.py  | Патерн Repository: CRUD-операції над кожною сутністю                            |
| database.py    | Налаштування SQLAlchemy Engine та Session; seed-функції                         |
| reports.py     | Функції формування звітів (середні бали тощо)                                   |
| cli/           | Підкаталог з CLI-модулями для кожної групи команд                               |
| tests/         | Автоматизовані тести та фікстури                                                |
| pyproject.toml | Конфігурація проєкту та залежності                                              |

### 2.2.1 Реалізація ООП-принципів у проєкті

**Абстракція.** Клас Person є абстрактним міксином, що визначає спільний контракт для всіх учасників навчального процесу. Він вимагає від підкласів реалізації методу `info()` та властивостей `name` і `phone`.

**Наслідування.** Класи `Student` і `Teacher` наслідують `Base` та `Person`. Клас `AbstractRepository` є базовим для конкретних репозиторіїв.

**Поліморфізм.** Метод `info()` визначено у `Person` як спільну вимогу, але кожен підклас реалізує його по-своєму: `Student.info()` повертає дані учня, а `Teacher.info()` — дані викладача зі спеціалізацією.

**Інкапсуляція.** Поля `_name`, `_phone`, `_specialization` є захищеними. Доступ до них відбувається через `@property` та сетери з базовою валідацією значень.

**Узагальнення (Generics).** `AbstractRepository` є параметризованим класом `AbstractRepository[T]`, що дозволяє писати типізований код без дублювання.

## 2.3 Структура підсистеми зберігання даних

База даних складається із семи таблиць, пов'язаних зовнішніми ключами.

**Таблиця `students`.** Зберігає ім'я учня, телефон і дату народження.

**Таблиця `teachers`.** Містить контактні дані викладача і його спеціалізацію.

**Таблиця `subjects`.** Зберігає перелік предметів.

**Таблиця `groups`.** Зберігає навчальні групи.

**Таблиця `group_students`.** Реалізує зв'язок many-to-many між групами й учнями.

**Таблиця `lessons`.** Описує заняття: дату й час, аудиторію, викладача, групу та предмет.

**Таблиця `grades`.** Зберігає оцінки учнів за конкретні заняття.

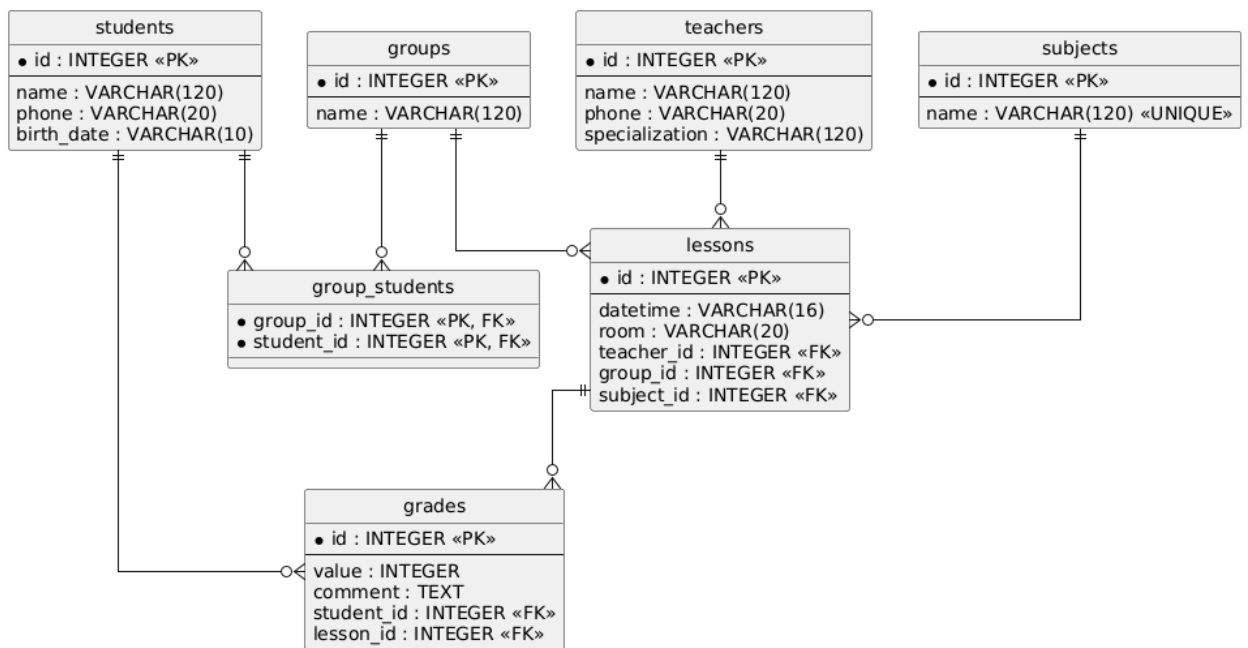


Рисунок 3: Схема бази даних облікової системи музичної школи

Таблиця 2.3 – Детальна структура таблиці lessons

| Поле       | Тип         | NULL | Опис                                  |
|------------|-------------|------|---------------------------------------|
| id         | INTEGER     | Hi   | Первинний ключ, автоінкремент         |
| datetime   | VARCHAR(16) | Hi   | Дата і час заняття (YYYY-MM-DD HH:MM) |
| room       | VARCHAR(20) | Hi   | Номер аудиторії                       |
| teacher_id | INTEGER     | Hi   | FK → teachers.id                      |
| group_id   | INTEGER     | Hi   | FK → groups.id                        |
| subject_id | INTEGER     | Hi   | FK → subjects.id                      |

## 2.4 Структура модулів програми

### 2.4.1 Модуль models.py

Містить 6 ORM-класів і 1 абстрактний міксин. Ієрархія класів:

Person (абстрактний міксин)

├─ Student (Base + Person)

└─ Teacher (Base + Person)

Base (DeclarativeBase)

└─ Student  
└─ Teacher  
└─ Subject  
└─ Group  
└─ Lesson  
└─ Grade

group\_students (Table – асоціативна таблиця)

Ключові методи:

- `Person.__init_subclass__()` – перевіряє контракт підкласів при оголошенні;
- `Student.info()` / `Teacher.info()` – поліморфний вивід інформації про особу;
- `Group.add_student()` / `Group.remove_student()` – управління складом групи;
- `Grade.grade_label()` – перетворення числової оцінки у словесну.

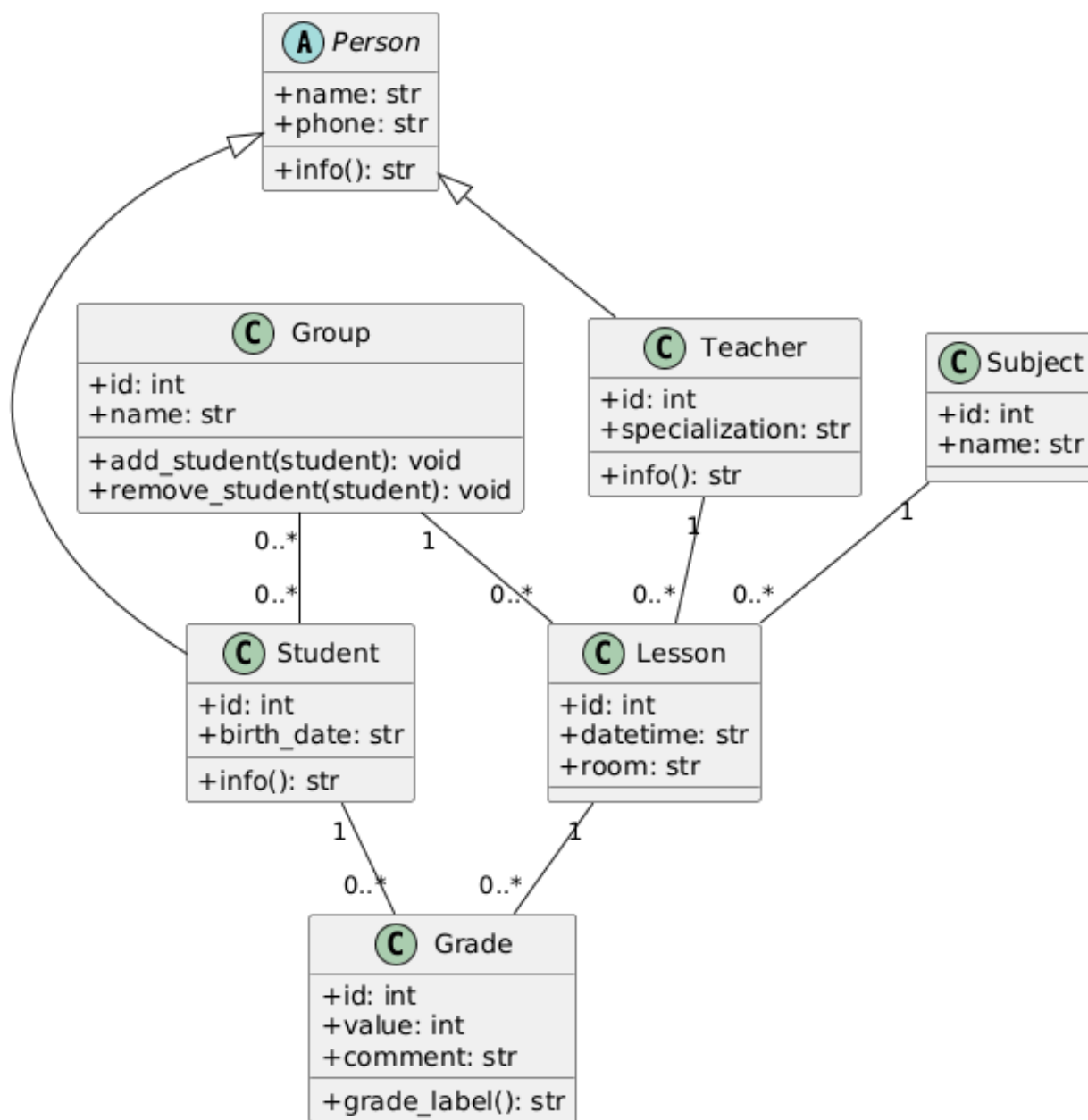


Рисунок 4: UML-діаграма класів предметної області

#### 2.4.2 Модуль `repository.py`

Модуль `repository.py` ізолює логіку доступу до даних від решти програми. Ієрархія має такий вигляд:

```
AbstractRepository[T] (ABC, Generic[T])
```

```
├─ StudentRepository
```

```
├─ TeacherRepository
```

```
├─ SubjectRepository
```

```
├─ GroupRepository
```

└ LessonRepository

└ GradeRepository

AbstractRepository реалізує чотири базові операції: `get(id)`, `get_all()`, `add(entity)`, `delete(id)`. Конкретні репозиторії доповнюють їх спеціалізованими методами:

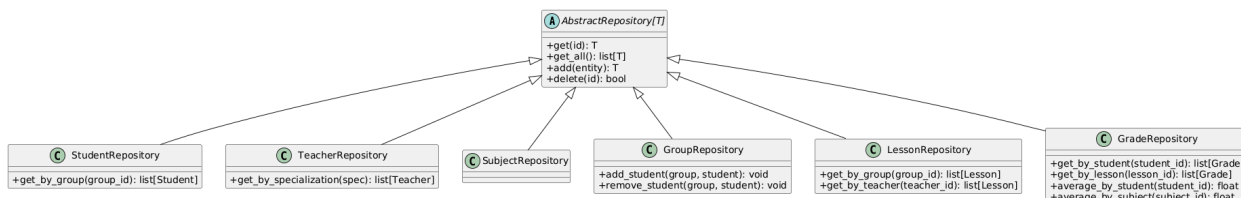


Рисунок 5: UML-діаграма класів репозиторіїв

Таблиця 2.4 — Специфічні методи репозиторіїв

| Репозиторій / Метод                            | Опис                                       |
|------------------------------------------------|--------------------------------------------|
| StudentRepository.get_by_group(group_id)       | Учні конкретної групи                      |
| TeacherRepository.get_by_specialization(spec)  | Пошук викладачів за спеціалізацією (ILIKE) |
| GroupRepository.add_student(group, student)    | Додати учня до групи                       |
| GroupRepository.remove_student(group, student) | Видалити учня з групи                      |
| LessonRepository.get_by_group(group_id)        | Заняття конкретної групи                   |
| LessonRepository.get_by_teacher(teacher_id)    | Заняття конкретного викладача              |
| GradeRepository.get_by_student(student_id)     | Всі оцінки учня                            |
| GradeRepository.get_by_lesson(lesson_id)       | Всі оцінки за заняття                      |
| GradeRepository.average_by_student(student_id) | Середній бал учня                          |
| GradeRepository.average_by_subject(subject_id) | Середній бал по предмету                   |

Лістинг 2.1 — Абстрактний репозиторій (repository.py, скорочено)

```
class AbstractRepository(ABC, Generic[T]):  
    model: Type[T]  
    default_order_by: tuple = ()  
  
    def __init__(self, session: Session) -> None:  
        self._session = session  
  
    def get(self, entity_id: int) -> Optional[T]:  
        return self._session.get(self.model, entity_id)  
  
    def get_all(self) -> list[T]:  
        stmt = select(self.model)  
        if self.default_order_by:  
            stmt = stmt.order_by(*self.default_order_by)  
        return self._scalar_list(stmt)  
  
    def add(self, entity: T) -> T:  
        self._session.add(entity)  
        self._session.flush()  
        return entity  
  
    def delete(self, entity_id: int) -> bool:  
        entity = self.get(entity_id)  
        if not entity:  
            return False  
        self._session.delete(entity)  
        return True
```

#### 2.4.3 Модуль database.py

Відповідає за ініціалізацію рушія SQLAlchemy, створення сесій і наповнення бази демонстраційними даними. Функція `seed_demo()` створює тестових учнів, викладачів, предмети, групи, заняття та оцінки, що дає змогу швидко перевірити основні можливості системи.

#### 2.4.4 Каталог `cli/`

Кожен файл каталогу реалізує групу команд Click для окремої сутності.

Наприклад, `cli/students.py` містить команди:

- `student list` — вивід усіх учнів;
- `student add` — інтерактивне додавання учня;
- `student show <id>` — деталі учня;
- `student edit <id>` — редагування;
- `student delete <id>` — видалення.

Лістинг 2.2 — Приклад команди CLI (`cli/grades.py`, спрощено)

```
@grade.command("add")
@click.argument("lesson_id", type=int)
@click.argument("student_id", type=int)
@click.option("--value", "-v", type=int, prompt="Оцінка (1-12)")
@click.option("--comment", "-c", default="", prompt="Коментар")
def grade_add(lesson_id, student_id, value, comment):
    """Виставити оцінку учню за заняття."""
    with get_session() as session:
        lesson_repo = LessonRepository(session)
        student_repo = StudentRepository(session)
        grade_repo = GradeRepository(session)

        lesson = lesson_repo.get(lesson_id)
        student = student_repo.get(student_id)
```

```

grade = Grade(student=student, lesson=lesson,
               value=value, comment=comment)

grade_repo.add(grade)

click.echo(f"✓ Оцінку виставлено: {grade}")

```

## 2.5 Опис інтерфейсу користувача

Інтерфейс програми є консольним. Користувач працює із системою через команди в терміналі, вводячи назву групи та потрібну підкоманду.

**Довідка.** Команда `python main.py --help` виводить перелік доступних груп, а довідка конкретної групи показує її підкоманди та параметри.

**Перегляд даних.** Команди типу `list` виводять короткі списки об'єктів у зручному для читання форматі.

```

> python main.py student list
[1] Іван Петренко | 099-111-22-33 | 2010-04-12
[2] Марія Коваль  | 097-555-44-11 | 2011-09-05
[3] Андрій Бойко  | 093-777-18-28 | 2009-12-20

```

Лістинг 1: Приклад виводу команди `student list` у терміналі

**Додавання даних.** Команди `add` працюють в інтерактивному режимі: система послідовно запитує потрібні поля і перевіряє коректність введення.

**Звіти.** Команди `report` формують підсумкову інформацію про успішність учнів і предметів.

Для друкованої версії курсової роботи до цього розділу доцільно додати реальні скріншоти терміналу з виконанням команд `--help`, `student list` та `report student 1`.

Таблиця 2.5 — Доступні групи команд CLI

| Група   | Призначення       | Приклад                   |
|---------|-------------------|---------------------------|
| student | Управління учнями | student list, student add |

|         |                        |                                      |
|---------|------------------------|--------------------------------------|
| teacher | Управління викладачами | teacher add, teacher show 1          |
| subject | Управління предметами  | subject list                         |
| group   | Управління групами     | group show 1, group add-student      |
| lesson  | Управління заняттями   | lesson add, lesson list --group-id 1 |
| grade   | Виставлення оцінок     | grade add, grade list-lesson 1       |
| report  | Звіти про успішність   | report student 1, report subject 2   |
| seed    | Наповнення демоданими  | seed demo                            |

```
> python main.py report student 1
Учень: Іван Петренко
Кількість оцінок: 6
Середній бал: 10.33
Рівень успішності: високий
```

Лістинг 2: Приклад виводу звіту про успішність учня

## 2.6 Результати тестування роботи програми

Тестування проводилося двома способами: автоматизованою перевіркою основних модулів і ручною перевіркою основних сценаріїв роботи програми.

**Автоматизовані тести.** Тести розташовані в каталозі tests/. Для запуску використовується команда:

```
uv run python -m unittest discover -s tests -p 'test_*.py'
```

Автоматизовані тести перевіряють:

- коректність CRUD-операцій репозиторіїв;
- виставлення та отримання оцінок;
- обрахунок середнього балу;
- валідацію оцінки (значення поза 1–12 викликає ValueError);

- роботу CLI-команд через CliRunner з Click.

**Ручне тестування.** Після наповнення системи демонстраційними даними були перевірені основні користувацькі сценарії: перегляд списків, додавання записів, показ деталей, формування звітів та обробка некоректних значень.

Таблиця 2.6 — Результати тестування основних сценаріїв

| №  | Сценарій                                               | Результат                   |
|----|--------------------------------------------------------|-----------------------------|
| 1  | Виведення списку учнів (student list)                  | ✓ Пройшов                   |
| 2  | Додавання нового викладача (teacher add)               | ✓ Пройшов                   |
| 3  | Перегляд складу групи (group show 1)                   | ✓ Пройшов                   |
| 4  | Виставлення оцінки учню (grade add)                    | ✓ Пройшов                   |
| 5  | Спроба виставити оцінку 13 (некоректна)                | ✓ Відхилено з повідомленням |
| 6  | Звіт середнього балу учня (report student 1)           | ✓ Пройшов                   |
| 7  | Звіт середнього балу по предмету (report subject 1)    | ✓ Пройшов                   |
| 8  | Видалення учня з каскадним видаленням оцінок           | ✓ Пройшов                   |
| 9  | Фільтрація занять за групою (lesson list --group-id 1) | ✓ Пройшов                   |
| 10 | Повторне наповнення бази без конфліктів                | ✓ Пройшов                   |

```
> uv run python -m unittest discover -s tests -p "test_*.py"
.....
-----
Ran 32 tests in 1.84s

OK
```

### Лістинг 3: Приклад результатів запуску автоматизованих тестів

Для остаточного оформлення роботи в цей розділ також бажано вставити реальний скріншот запуску тестів або приклад ручної перевірки однієї з CLI-команд.

## ВИСНОВКИ

У результаті виконання курсової роботи розроблено консольну облікову систему музичної школи, яка охоплює основні процеси ведення навчальної діяльності.

Відповідно до технічного завдання виконано:

- спроектовано доменну модель з урахуванням усіх принципів ООП: абстракція (клас `Person`), наслідування (`Student`, `Teacher` від `Person` та `Base`), поліморфізм (метод `info()`), інкапсуляція (приватні поля через `@property`);
- реалізовано сім взаємопов'язаних таблиць бази даних `SQLite` через `SQLAlchemy 2.0 ORM` з підтримкою зовнішніх ключів, каскадного видалення та обмежень цілісності;
- реалізовано окремий рівень доступу до даних, що спрощує підтримку коду та підвищує зручність тестування;
- побудовано зручний CLI-інтерфейс за допомогою `Click` з інтерактивним введенням, фільтрацією та формуванням звітів;
- написано автоматизовані тести, що перевіряють коректність основних операцій системи.

Система успішно пройшла тестування: усі заплановані сценарії дали очікуваний результат, а некоректні введення обробляються належним чином.

Практичне значення роботи полягає у можливості використання системи для щоденного обліку даних у музичній школі, а також як навчального прикладу побудови CLI-застосунків на `Python` з використанням `ORM`.

## СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Python Software Foundation. Python 3.12 Documentation. URL: <https://docs.python.org/3.12/>.
2. SQLAlchemy Documentation. SQLAlchemy 2.0 ORM Quickstart. URL: <https://docs.sqlalchemy.org/en/20/orm/quickstart.html>.
3. Click Documentation. Pallets Projects. URL: <https://click.palletsprojects.com/en/8.x/>.
4. SQLite Documentation. About SQLite. URL: <https://www.sqlite.org/about.html>.
5. uv Documentation. Astral. URL: <https://docs.astral.sh/uv/>.
6. Python unittest Documentation. URL: <https://docs.python.org/3/library/unittest.html>.
7. SQLAlchemy ORM. Mapped column. URL: [https://docs.sqlalchemy.org/en/20/orm/mapping\\_columns.html](https://docs.sqlalchemy.org/en/20/orm/mapping_columns.html).
8. Real Python. Python's property(): Add Managed Attributes to Your Classes. URL: <https://realpython.com/python-property/>.
9. Typst Documentation. URL: <https://typst.app/docs/>.
10. Anthropic. Claude. URL: <https://claude.ai/>.
11. OpenAI. Codex. URL: <https://openai.com/codex/>.
12. GitHub repository: sioxy/music\_school. URL: [https://github.com/sioxy/music\\_school](https://github.com/sioxy/music_school).

## ДОДАТКИ

### Додаток А. Код модуля `models.py` (фрагмент)

Лістинг А.1 — Клас `Student` (`models.py`)

```
class Student(Base, Person):
    """Учень школи. Наслідує Base (ORM) + Person (доменна
абстракція)."""
    __tablename__ = "students"

    id: Mapped[int] = mapped_column(
        Integer, primary_key=True, autoincrement=True)
    _name: Mapped[str] = mapped_column(
        "name", String(120), nullable=False)
    _phone: Mapped[str] = mapped_column(
        "phone", String(20), nullable=False)
    birth_date: Mapped[str] = mapped_column(
        String(10), nullable=False)

    groups: Mapped[list[Group]] = relationship(
        "Group", secondary=group_students,
        back_populates="students")
    grades: Mapped[list[Grade]] = relationship(
        "Grade", back_populates="student",
        cascade="all, delete-orphan")

    @property
    def name(self) -> str:
        return self._name
```

```

@name.setter
def name(self, value: str) -> None:
    self._name = value.strip()

def info(self) -> str:
    return (
        f"[Учень #{self.id}] {self._name} "
        f"| Народився: {self.birth_date} "
        f"| Тел.: {self._phone}"
    )

```

#### Додаток Б. Код модуля repository.py (фрагмент)

Лістинг Б.1 — GradeRepository з методами розрахунку середнього балу

```

class GradeRepository(AbstractRepository[Grade]):
    model = Grade

    def get_by_student(self, student_id: int) -> list[Grade]:
        return self._scalar_list(
            select(self.model)
                .where(self.model.student_id == student_id)
                .order_by(self.model.lesson_id)
        )

    def average_by_student(
        self, student_id: int) -> Optional[float]:
        result = self._session.execute(
            select(func.avg(self.model.value))
                .where(self.model.student_id == student_id)
        ).scalar()
        return float(result) if result is not None else None

```

```

def average_by_subject(
    self, subject_id: int) -> Optional[float]:
    result = self._session.execute(
        select(func.avg(self.model.value))
        .join(self.model.lesson)
        .where(Lesson.subject_id == subject_id)
    ).scalar()
    return float(result) if result is not None else None

```

## Додаток В. Структура проєкту

music\_school/

```

├─ main.py          # Точка входу CLI
├─ models.py        # ORM-моделі (Student, Teacher, ...)
├─ repository.py    # Патерн Repository
├─ database.py      # SQLAlchemy Engine + seed
├─ reports.py       # Функції звітності
├─ pyproject.toml   # Залежності проєкту
├─ requirements.txt  # Список залежностей (pip)
├─ uv.lock          # Lock-файл uv
├─ .python-version  # Версія Python (3.12)
├─ .gitignore
├─ cli/             # CLI-команди (Click)
|   ├─ __init__.py
|   ├─ students.py
|   ├─ teachers.py
|   ├─ subjects.py
|   ├─ groups.py
|   ├─ lessons.py
|   └─ grades.py

```

```
|   └─ reports.py
|   └─ seed.py
└─ tests/                # Автоматизовані тести
    └─ __init__.py
    └─ fixtures.py
    └─ test_*.py
```